

Experiment 8

Multirate Signal Processing

Date: 8.1 Implementation and Analysis of Decimation and Interpolation in Multirate Signal Processing Using MATLAB

Program Code

```
clc;

clear;

close all;

% Original Sequence

n = 0:40;

x = sin(0.3*pi*n);

% Decimation by 2

y = x(1:2:end);

nd = 0:length(y)-1;

% Interpolation by 2

v = zeros(1,2*length(x));

v(1:2:end) = x;

ni = 0:length(v)-1;

% Figure 1: Original Sequence

figure;

stem(n,x,'filled');

grid on;

title('Original Sequence x[n]');

xlabel('n');

ylabel('Amplitude');

% Figure 2: Decimated Sequence

figure;

stem(nd,y,'filled');
```

```

grid on;

title('Decimated Sequence by 2');

xlabel('n');

ylabel('Amplitude');

% Figure 3: Interpolated Sequence

figure;

stem(ni,v,'filled');

grid on;

title('Interpolated Sequence by 2');

xlabel('n');

ylabel('Amplitude');

% Figure 4: Comparison

figure;

stem(n,x,'b'); hold on;

stem(0:2:length(v)-2,v(1:2:end),'r');

legend('Original x[n]','Interpolated Samples');

grid on;

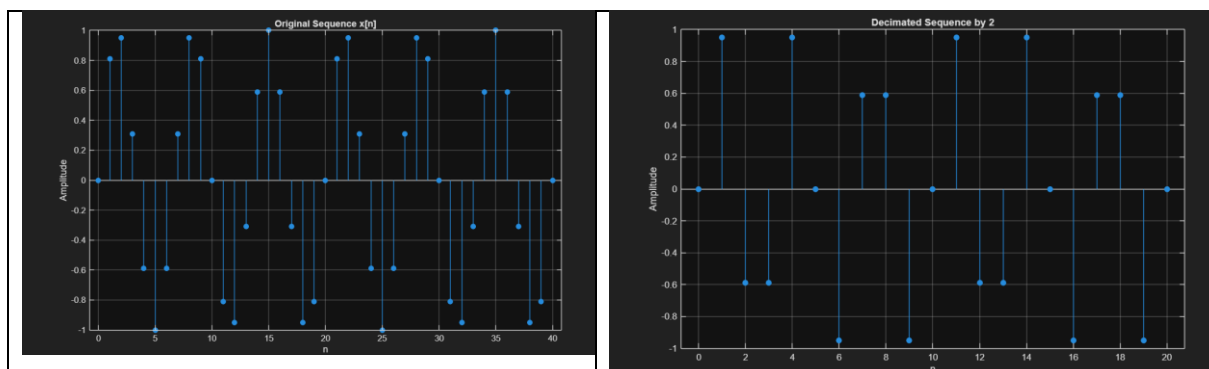
title('Comparison of Sequences');

xlabel('n');

ylabel('Amplitude');

```

OUTPUT



8.2 Polyphase Implementation of Decimation and Interpolation Filters Using MATLAB

Program Code

```
clc;

clear;

close all;

%% Input Signal

Fs = 1000; % Sampling Frequency

t = 0:1/Fs:1; % Time Vector

% Test Signal

x = sin(2*pi*50*t) + 0.5*sin(2*pi*120*t);

%% FIR Low-Pass Filter Design

N = 20; % Filter Order

Fc = 0.4; % Normalized Cutoff Frequency

h = fir1(N,Fc,hamming(N+1));

%% Decimation Factor

M = 2;

% Polyphase Decimation

y_dec = upfirdn(x,h,1,M);

%% Interpolation Factor

L = 2;

% Polyphase Interpolation

y_int = upfirdn(y_dec,h,L,1);

%% Adjust Length for Comparison

if length(y_int) > length(x)

y_int = y_int(1:length(x));

elseif length(y_int) < length(x)
```

```

x = x(1:length(y_int));

end

%% Time Axes

t1 = (0:length(x)-1)/Fs;

t2 = (0:length(y_dec)-1)/(Fs/M);

t3 = (0:length(y_int)-1)/Fs;

%% BEST GRAPH OUTPUT

figure('Name','Polyphase Decimation and Interpolation','NumberTitle','off');

% Original Signal

subplot(4,1,1)

plot(t1,x,'b','LineWidth',1.5)

title('Original Input Signal')

xlabel('Time (s)')

ylabel('Amplitude')

grid on

% Decimated Signal

subplot(4,1,2)

stem(t2,y_dec,'r','filled')

title('Decimated Signal (M = 2)')

xlabel('Time (s)')

ylabel('Amplitude')

grid on

% Interpolated Signal

subplot(4,1,3)

plot(t3,y_int,'k','LineWidth',1.5)

title('Interpolated/Reconstructed Signal (L = 2)')

xlabel('Time (s)')

ylabel('Amplitude')

```

```

grid on

% Comparison
subplot(4,1,4)
plot(t1,x,'b','LineWidth',1.5)

hold on
plot(t3,y_int,'r--','LineWidth',1.5)

title('Comparison of Original and Reconstructed Signals')
xlabel('Time (s)')
ylabel('Amplitude')
legend('Original Signal','Reconstructed Signal')

grid on

%% Display Information
disp('-----');
disp('POLYPHASE DECIMATION & INTERPOLATION');
disp('-----');
fprintf('Original Signal Length = %d\n',length(x));
fprintf('Decimated Signal Length = %d\n',length(y_dec));
fprintf('Interpolated Signal Length = %d\n',length(y_int));

%% Separate Figure for Better Visualization
figure;
plot(t1,x,'b','LineWidth',2)

hold on
plot(t3,y_int,'r--','LineWidth',2)

title('Original vs Reconstructed Signal')
xlabel('Time (s)')
ylabel('Amplitude')
legend('Original Signal','Reconstructed Signal')

grid on

```

OUTPUT

